Cheat sheet

Spatie Laravel Event Sourcing Cheat Sheet

A dense, printable reference for spatie/laravel-event-sourcing: aggregates, projectors, reactors, artisan commands, testing, and the gotchas people hit first.

Covers spatie/laravel-event-sourcing v7 · Last reviewed 2026-08-07

Not an event sourcing tutorial. This assumes you've already read the docs or built one aggregate and are past "what is an event" — it's the page you keep open while building, plus the mistakes that don't show up until production. Still deciding whether event sourcing is worth the complexity? Read the callout below first.

WHEN NOT TO USE EVENT SOURCING

Reach for it only when history, audit, or genuinely complex invariants matter. For plain CRUD with no temporal requirement, it is ceremony you pay for on every future change, not a default.

THE FLOW

```
command
  ↓
   aggregate root (guards + recordThat)
     ↓
      persist()
        ↓
         stored_events
           ↓
             projectors build read models / reactors fire side effects
```

BUILDING BLOCKS

Aggregate root: Guards business rules, records events, rebuilt from its event stream. Extends AggregateRoot.

Event: An immutable fact that happened. Implements ShouldBeStored.

Projector: Builds/updates read models from events. Replayable, side-effect-free.

Reactor: Performs side effects (mail, notifications, HTTP). Not replayed.

Read model / projection: The queryable state projectors maintain.

StoredEvent: The persisted event row in the event store.

Snapshot: Cached aggregate state to avoid replaying long streams.

AGGREGATE ROOT ESSENTIALS

```
AggregateRoot::retrieve($uuid)
```

Rebuild from history.

```
->recordThat(new SomethingHappened(...))
```

Append an event.

```
->persist()
```

Write recorded events to the store.

```
protected function applySomethingHappened(SomethingHappened $event)
```

Mutate in-memory state (no side effects, no queries into other aggregates).

Enforce invariants BEFORE recordThat; throw domain exceptions on violation.

Concurrency is built in: persist() throws CouldNotPersistAggregate if another process persisted events for this aggregate since it was retrieved. No manual version passing needed.

PROJECTORS VS REACTORS (THE RULE PEOPLE GET WRONG)

Projectors: idempotent, replay-safe, only touch their read models.

Side effects (emails, external calls) go in reactors, never projectors, because a replay would re-fire them.

HANDLERS

```
public function onSomethingHappened(SomethingHappened $event)
```

Projector/reactor handler method. Name is derived from the event's short class name — rename the event and the handler silently stops firing.

```
'projectors' ⇒ [OrderProjector::class],
'reactors' ⇒ [OrderReactor::class],
```

config/event-sourcing.php — registers handlers for every request. Preferred default.

```
Projectionist::addProjector(OrderProjector::class);
```

Runtime registration from a service provider's boot() — for conditional handlers (feature flags, tenants), not the common case.

VERSIONING EVENTS (UPCASTING)

An event's shape is frozen the moment it's stored. Add a field, rename a field, split an event — old rows on disk still have the old shape, and replay will hydrate them into the current class.

```
class UpcastingEventSerializer extends JsonEventSerializer {
    public function deserialize(string $eventClass, string $json, array $metadata): ShouldBeStored {
        $payload = json_decode($json, true);
        if ($eventClass ⩵ AddressChanged::class && !isset($payload['country'])) {
            $payload['country'] = 'unknown';
        }
        return parent::deserialize($eventClass, json_encode($payload), $metadata);
    }
}
```

Decorate the default serializer and patch the raw payload before it's hydrated, keyed off the fields that are actually missing.

```
'event_serializer' ⇒ UpcastingEventSerializer::class,
```

config/event-sourcing.php — swap in the decorated serializer.

Do this before you need it. Retrofitting an upcaster after three shapes have shipped means branching on three payload versions in one method.

ARTISAN COMMANDS

```
php artisan make:aggregate
```

Scaffold a new aggregate root.

```
php artisan make:projector
```

Scaffold a new projector. Add -Q for a QueuedProjector.

```
php artisan make:reactor
```

Scaffold a new reactor.

```
php artisan make:storable-event
```

Scaffold a new domain event.

```
php artisan event-sourcing:replay
```

Replay stored events to projectors.

```
php artisan event-sourcing:list
```

List all registered event handlers.

QUEUED PROJECTORS AND FAILURES

```
php artisan make:projector OrderProjector -Q
```

Runs the projector on a queue instead of synchronously in the request.

A queued projector that throws fails like any other queued job: it retries per your queue config, then lands on failed_jobs. The command that recorded the event already succeeded, this does not roll it back.

A failed queued projector leaves its read model behind whatever events it did process before failing. Re-running event-sourcing:replay for that projector after fixing the bug is how you catch it back up, not artisan queue:retry.

QUERYING THE EVENT STORE

```
StoredEvent::query()->where('aggregate_uuid', $uuid)->orderBy('id')->get()
```

The full recorded history for one aggregate, in order. This is what AggregateRoot::retrieve() replays internally.

Use AggregateRoot::retrieve() when you just need current state. Query StoredEvent directly for an audit trail, a debug view, or an admin "what happened to this order" screen.

TESTING

```
AggregateRoot::fake($uuid)->given([...])->when(fn($agg) => ...)->assertRecorded([new Expected(...)])
```

Also see →assertNotRecorded(…).

GOTCHAS

Projectors must be idempotent and replay-safe. event-sourcing:replay runs every event through them again from scratch.

Don’t query other aggregates inside apply*. It reads live state at replay time, not the state that existed when the event was recorded.

Snapshot long-lived aggregates. Without one, retrieve() replays the entire stream on every load.

Wrong / right

Side effects inside a projector

```
class OrderProjector extends Projector
{
    public function onOrderShipped(OrderShipped $event)
    {
        Order::find($event->orderId)->update(['status' => 'shipped']);
        Mail::to($event->email)->send(new OrderShippedMail);
    }
}
```

Replay this once and every past customer gets re-emailed.

```
class OrderProjector extends Projector
{
    public function onOrderShipped(OrderShipped $event)
    {
        Order::find($event->orderId)->update(['status' => 'shipped']);
    }
}

class OrderReactor extends Reactor
{
    public function onOrderShipped(OrderShipped $event)
    {
        Mail::to($event->email)->send(new OrderShippedMail);
    }
}
```

Reactors aren’t replayed. Side effects live there, full stop.

Reading other aggregates inside apply*

```
protected function applyItemAdded(ItemAdded $event)
{
    $price = Product::retrieve($event->productId)->currentPrice();
    $this->total += $price;
}
```

Replay this next year and you’ll total the order at today’s prices.

```
public function addItem(string $productId, int $priceCents)
{
    $this->recordThat(new ItemAdded($productId, $priceCents));
}

protected function applyItemAdded(ItemAdded $event)
{
    $this->total += $event->priceCents;
}
```

Decide the price when the command runs, store it in the event. apply* only touches what’s already there.

Invariant checks after recordThat

```
public function ship()
{
    $this->recordThat(new OrderShipped);

    if ($this->status !== 'paid') {
        throw new CannotShipUnpaidOrder;
    }
}
```

recordThat() doesn’t roll back. The invalid event is already in the stream.

```
public function ship()
{
    if ($this->status !== 'paid') {
        throw new CannotShipUnpaidOrder;
    }

    $this->recordThat(new OrderShipped);
}
```

Guard first, record second. Always.

create() vs updateOrCreate() on replay

```
protected function onOrderPlaced(OrderPlaced $event)
{
    Order::create([
        'id' ⇒ $event→orderId,
        // ...
    ]);
}
```

event-sourcing:replay throws a duplicate-key error the second time it runs.

```
protected function onOrderPlaced(OrderPlaced $event)
{
    Order::updateOrCreate(
        ['id' ⇒ $event→orderId],
        [/* ... */]
    );
}
```

updateOrCreate() makes the projector safe to replay from an empty read model.